VITyarthi – Build Your Own Project

Student Performance & Attendance Management System

Python + SQLite Project

Student Name: _____

Registration Number: _____

Course / Subject: _____

Faculty Name: _____

Academic Year: 2026–27

1. Introduction

The Student Performance & Attendance Management System is a modular Python application for maintaining basic academic records. It stores student details, subject-wise marks and attendance in an SQLite database and generates individual and class-level performance reports.

2. Problem Statement

Manual management of student records, marks and attendance can require repeated calculations and can make it difficult to obtain a consolidated academic view. The proposed system provides a simple database-backed solution with validation and automated calculations.

3. Objectives

The objectives are to manage student records, store marks and attendance, calculate performance indicators, generate reports, demonstrate modular Python programming, use persistent storage, and apply validation and testing.

4. Functional Requirements

FR1 Add student records. FR2 View student records. FR3 Delete student records. FR4 Add subject-wise marks. FR5 View marks. FR6 Add subject-wise attendance. FR7 View attendance percentages. FR8 Generate individual performance reports. FR9 Generate class performance reports.

5. Non-Functional Requirements

Usability: menu-driven interaction. Reliability: validation prevents invalid academic data. Maintainability: separate modules provide focused responsibilities. Performance: lightweight SQLite queries are appropriate for a small academic dataset. Error handling: invalid operations are caught and reported. Resource efficiency: the application uses the Python standard library and SQLite without external runtime packages.

6. System Architecture

The application is organized into a user-interface layer, functional management modules, an analytics/reporting layer, and a SQLite persistence layer. main.py coordinates the user interaction. Student, marks and attendance modules manage domain operations. Analytics calculates aggregate indicators, while report_generator formats results.

7. Design Diagrams

The project package contains Mermaid source for the system architecture, workflow, use-case diagram, sequence diagram, class/component diagram and ER diagram in docs/diagrams.md. These diagrams describe the relationships and flow of the implemented system.

8. Database Design

The database contains three related entities: STUDENTS, MARKS and ATTENDANCE. A student can have multiple marks records and multiple attendance records. Foreign keys connect academic records to the corresponding student.

9. Design Decisions & Rationale

Python was selected for clear modular implementation and because the project applies programming concepts. SQLite was selected for simple persistent relational storage without requiring a separate server. The code is divided by responsibility to improve maintainability and testing.

10. Implementation Details

The source contains database.py, student_manager.py, marks_manager.py, attendance_manager.py, analytics.py, report_generator.py and main.py. Input constraints are checked before database insertion. SQLite constraints provide an additional data-integrity layer.

11. Screenshots / Results

The final submitted version should include screenshots of the main menu, student insertion, marks entry, attendance entry, an individual performance report and a class performance report. The project package includes demo_data.py to make demonstration data easy to load.

12. Testing Approach

Automated tests are implemented with Python unittest. Tests cover successful student creation, rejection of invalid marks, and correct calculation of average marks, attendance percentage and performance status. Run: python -m unittest discover -s tests -v.

13. Challenges Faced

Key implementation challenges include designing related database tables, validating user input, coordinating multiple modules, calculating aggregate metrics and maintaining a simple interface while keeping the implementation modular.

14. Learnings & Key Takeaways

The project demonstrates modular Python design, relational database concepts, validation, error handling, SQL aggregation, automated testing and technical documentation.

15. Future Enhancements

Possible enhancements include a Tkinter GUI, role-based login, CSV/PDF report export, graphical dashboards, richer subject analytics and cloud deployment.

16. References

Python documentation; SQLite documentation; VITyarthi Build Your Own Project requirements supplied for this submission.

Appendix A – Functional Modules

Module	Purpose
Student Management	Student CRUD operations
Marks Management	Subject-wise marks storage/retrieval
Attendance Management	Attendance storage and percentage
Analytics & Reporting	Performance calculations and reports